

CS310 Lab: Sorting Algorithms — Report

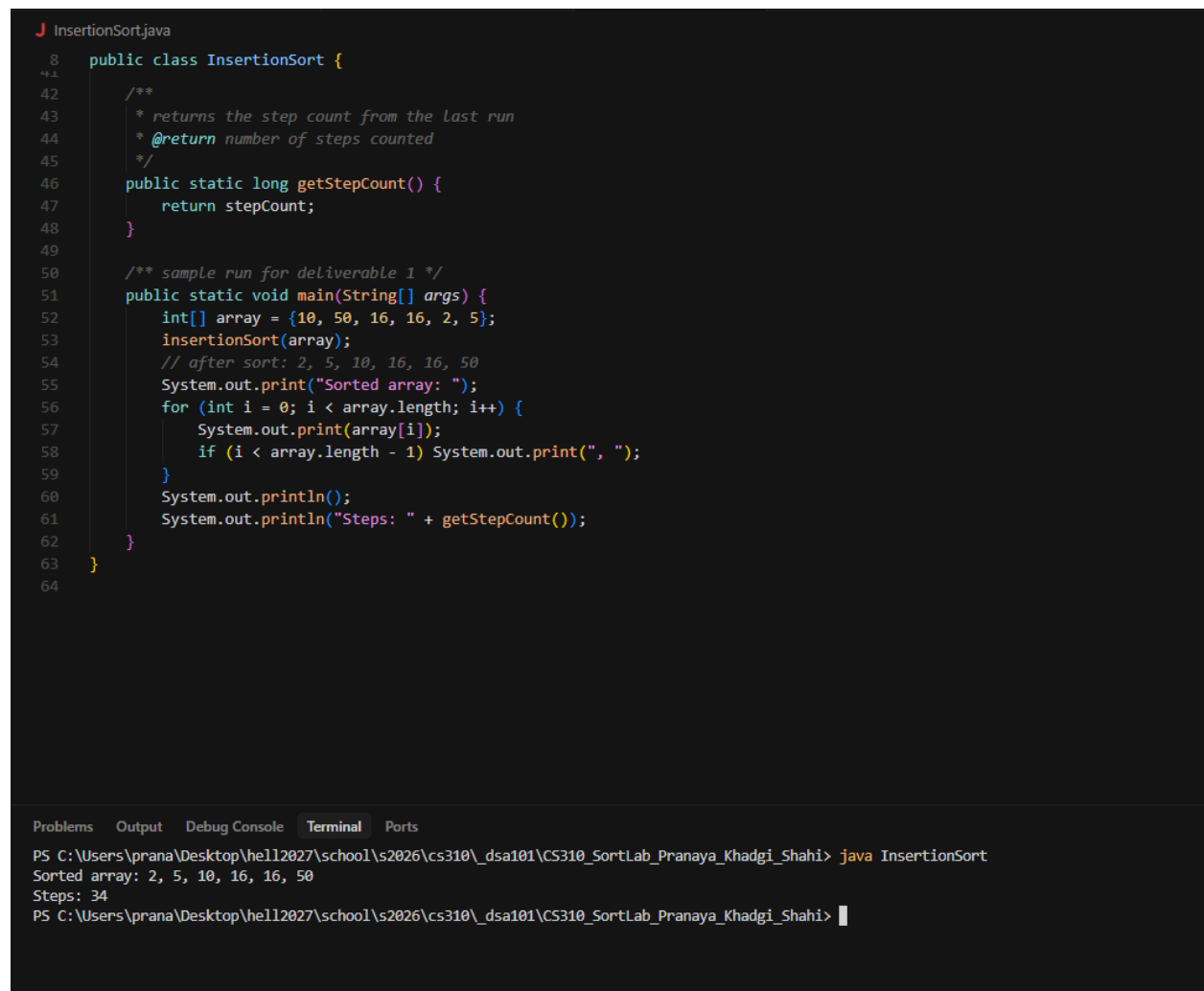
Author: Pranaya Khadgi Shahi

Date: February 2026

Deliverable 1: Four sorting programs and sample runs

I implemented the four sorting algorithms (insertion, selection, merge, quick) each in its own file. Each one has a main method that runs on the sample array {10, 50, 16, 16, 2, 5} and prints the sorted array and the step count. Below are the four screenshots showing the output of each program when I run it from the command line.

1. InsertionSort



```
J InsertionSort.java
8 public class InsertionSort {
9
10
42 /**
43  * returns the step count from the last run
44  * @return number of steps counted
45  */
46 public static long getStepCount() {
47     return stepCount;
48 }
49
50 /** sample run for deliverable 1 */
51 public static void main(String[] args) {
52     int[] array = {10, 50, 16, 16, 2, 5};
53     insertionSort(array);
54     // after sort: 2, 5, 10, 16, 16, 50
55     System.out.print("Sorted array: ");
56     for (int i = 0; i < array.length; i++) {
57         System.out.print(array[i]);
58         if (i < array.length - 1) System.out.print(", ");
59     }
60     System.out.println();
61     System.out.println("Steps: " + getStepCount());
62 }
63 }
64
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> java InsertionSort
Sorted array: 2, 5, 10, 16, 16, 50
Steps: 34
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi>
```

2. SelectionSort

```
J SelectionSort.java
1  /*
2   * CS310 Lab: Selection Sort Implementation
3   * @author Pranaya Khadgi Shahi
4   * @date 2/19/2026
5   *
6   * This program implements selection sort with step counting for the lab
7   */
8  public class SelectionSort {
9
10     // static counter for the number of steps
11     private static long stepCount = 0;
12
13     /**
14      * Sorts the given array in ascending order using selection sort.
15      * @param arr the array to sort (modified in place)
16      */
17     public static void selectionSort(int[] arr) {
18         stepCount = 0; // resets the count for each run
19
20         int n = arr.length;
21
22         for (int i = 0; i < n - 1; i++) {
23             int minIdx = i;
24             for (int j = i + 1; j < n; j++) {
25                 stepCount++; // comparison
26                 if (arr[j] < arr[minIdx]) {
27                     minIdx = j;
28                 }
29             }
30             if (minIdx != i) {
31                 int temp = arr[i];
32                 arr[i] = arr[minIdx];
33                 arr[minIdx] = temp;
34                 stepCount += 2; // swap (two assignments)
35             }
36         }
37     }
38 }

Problems Output Debug Console Terminal Ports
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> java SelectionSort
Sorted array: 2, 5, 10, 16, 16, 50
Steps: 21
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> 
```

3. MergeSort

```
J MergeSort.java
1  /*
2   * CS310 Lab: Merge Sort Implementation
3   * @author Pranaya Khadgi Shahi
4   * @date 2/19/2026
5   *
6   * implements merge sort with step counting to analyze the algorithms performance
7   */
8  public class MergeSort {
9
10     // static counter for number of steps required
11     private static long stepCount = 0;
12
13     /**
14      * Sorts the given array in ascending order using merge sort.
15      * @param arr the array to sort (modified in place)
16      */
17     public static void mergeSort(int[] arr) {
18         stepCount = 0; // resets count for each run
19         mergeSort(arr, 0, arr.length - 1);
20     }
21
22     /**
23      * Recursive merge sort on the range [begin, end].
24      */
25     private static void mergeSort(int[] arr, int begin, int end){
26         if (begin >= end) return;
27
28         stepCount++;
29         int mid = begin + (end - begin) / 2;
30         mergeSort(arr, begin, mid);
31         mergeSort(arr, mid + 1, end);
32         merge(arr, begin, mid, end);
33     }
34
35     // Problems Output Debug Console Terminal Ports
36     PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> java MergeSort
37     Sorted array: 2, 5, 10, 16, 16, 50
38     Steps: 48
39     PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> []
```

4. QuickSort

```
1  /*
7  */
8  public class QuickSort {
9
10     // static counter for the number of steps required
11     private static long stepCount = 0;
12
13     /**
14      * Sorts the given array in ascending order using quick sort.
15      * @param arr the array to sort (modified in place)
16      */
17     public static void quickSort(int[] arr) {
18         stepCount = 0; // reset for each run
19         quickSort(arr, 0, arr.length - 1);
20     }
21
22     /**
23      * Recursive quick sort on the range [low, high]. Uses last element as pivot.
24      */
25     private static void quickSort(int[] arr, int low, int high){
26         if (low < high) {
27             stepCount++;
28             int pivotIdx = partition(arr, low, high);
29             quickSort(arr, low, pivotIdx - 1);
30             quickSort(arr, pivotIdx + 1, high);
31         }
32     }
33
34     /**
35      * Partitions the array so that elements <= pivot are left, pivot in place.
36      * @return index of the pivot after partition
37      */
38     private static int partition(int[] arr, int low, int high) {
39         int pivot = arr[high];
```

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> java QuickSort
Sorted array: 2, 5, 10, 16, 16, 50
Steps: 33
PS C:\Users\prana\Desktop\hell2027\school\s2026\cs310\_dsa101\CS310_SortLab_Pranaya_Khadgi_Shahi> |
```

Deliverable 2: Simulation table and graphs

How I did the simulation

For each input size $N = 1K, 5K, 10K, 15K, 20K$ I did three things. For random values I generated N random numbers in the range 0 to N , ran each algorithm 1000 times, and took the average of the steps. For ascending order I first generated random numbers then sorted them in ascending order (using a separate sort method as the lab said we could), then ran each algorithm once on that sorted array and recorded the steps. For descending I did the same but sorted in descending order first. So the table has for each size the average steps for random (over 1000 runs), and the steps for one run on ascending and one run on descending.

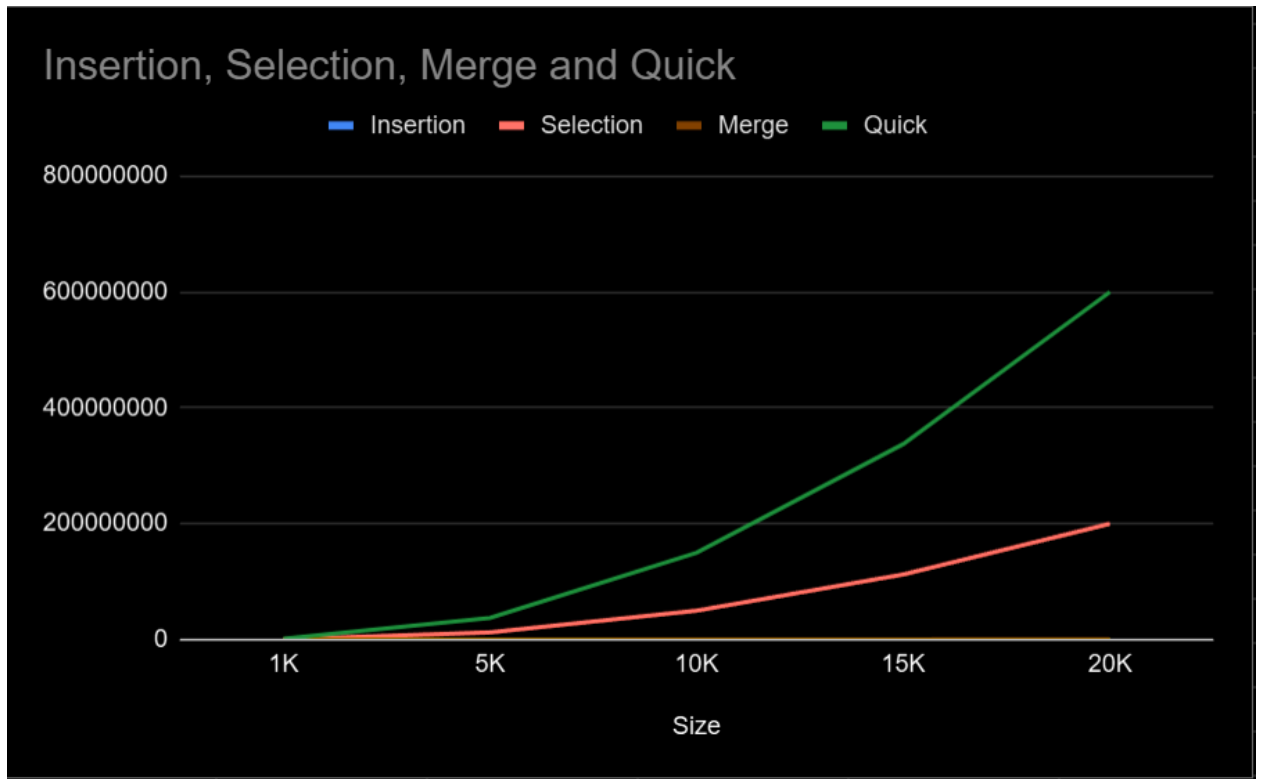
Table: Average steps by input size and input order

(Order in each cell: Insertion / Selection / Merge / Quick.)

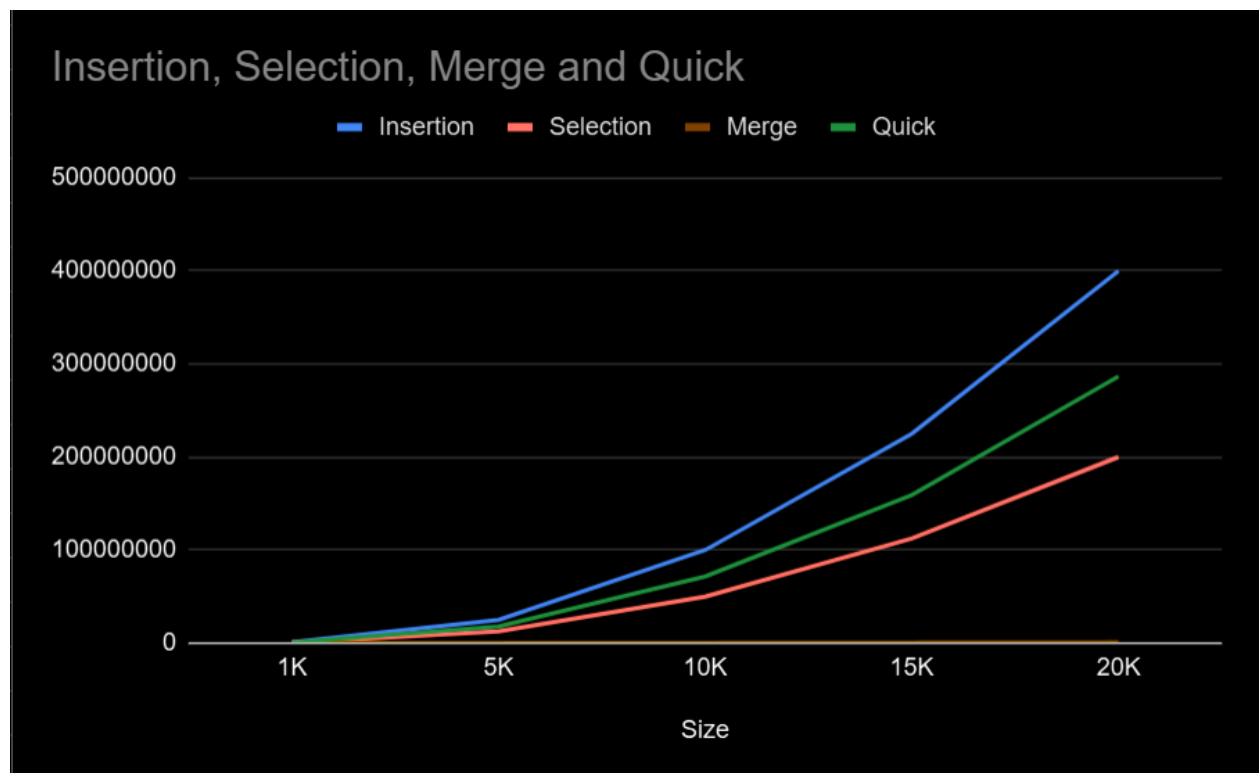
Input size	Ascending (avg steps)	Descending (avg steps)	Random (avg steps)
	Ins Sel Merge Quick	Ins Sel Merge Quick	Ins Sel Merge Quick
1K	2997 499500 25995 1501497	1000381 500746 26084 741900	501622 501483 29658 24518
5K	14997 12497500 160619 37507497	25001852 12503544 159699 17813710	12515442 12507480 183842 154990
10K	29997 49995000 346239 150014997	100003317 50007310 344543 71636190	49999925 50014979 397680 337489
15K	44997 112492500 538595 337522497	225005484 112510842 537832 159237430	112528293 112522478 621531 531074
20K	59997 199990000 742479 600029997	400007259 200014470 739010 286734709	199989794 200029978 855361 729097

I made two graphs. Here's how you can do it in **Excel** or **Google Sheets**.

Graph 1 — Already sorted (ascending)



Graph 2 — Sorted in descending order



Deliverable 3: Analysis (answers to the four questions)

(a) When elements are already sorted (ascending), which algorithm's performance is the worst? What is the reason?

From the data I got, quick sort was the worst when the input is already sorted in ascending order. In my implementation I used the last element as the pivot. So when the array is already sorted, the last element is always the biggest and every time we partition we get really unbalanced splits—almost everything goes to the left and nothing to the right. So the recursion goes really deep and we end up doing something like $O(N^2)$ comparisons. Insertion sort does well here because each element is already in place so it's like $O(N)$. Merge sort is $O(N \log N)$ either way. So the worst one in this case is quick sort and the reason is basically that picking the last element as pivot on sorted data gives the worst case.

(b) When elements are sorted in descending order, which algorithm's performance is the worst? What is the reason?

When the input is sorted in descending order, insertion sort was the worst in my results. For insertion sort, every new element you look at is smaller than everything you've already "sorted", so you have to compare and shift it all the way to the front every time. So you get a lot of comparisons and shifts, like $O(N^2)$ with a big constant. Selection sort is also $O(N^2)$ but it doesn't do as many shifts. Merge sort stays $O(N \log N)$. So insertion sort on descending input was the worst for me, and the reason is that each insertion has to move the element through the whole sorted part.

(c) Considering all the tabular data, which sorting algorithm's performance on average is better than the others? Why?

Looking at the whole table, merge sort had the best performance on average. Its step count goes up like $N \log N$ as N gets bigger, and it does that for random, ascending, and descending—there's no bad case. Quick sort can blow up when the input is sorted (like we saw in (a)). Insertion and selection both grow like N^2 so they get bad for larger sizes. So overall merge sort's numbers stayed the lowest for big N and it didn't have any weird spikes. The reason is that merge sort's worst case and average case are both $O(N \log N)$, so it's consistent.

(d) Considering all the tabular data, which sorting algorithm's performance on average is worse than the others? Why?

On average over all the data, insertion sort and selection sort did the worst, and usually insertion was a bit worse in terms of total steps. Both of them are $O(N^2)$ —insertion does a lot of comparisons and shifts especially on random and descending, and selection always does around $N^2/2$ comparisons no matter what. So when you look at the table for 10K, 15K, 20K their numbers get really high compared to merge and quick sort which grow like $N \log N$. So the worst average performance is from insertion (and selection), and the reason is they're both quadratic so they can't avoid doing on the order of N^2 operations, while the other two scale better.

End of report.